

Structure de données

Structures

DR. HASNA ELALAOUI

h.elalaoui@uca.ac.ma

Structures

Détails sur la partie

- ❑ Contexte général
- ❑ Utilisation des structures de données
- ❑ Manipulation des structures
 - ❑ Opérations sur les structures
 - ❑ Structures et tableaux
 - ❑ Structures et pointeurs
 - ❑ Structures et fonctions
 - ❑ Structures imbriquées
- ❑ Enumérations

Contexte Général

Problématique

- ❑ Les tableaux sont une structure qui permet de stocker plusieurs valeurs **de même type**.
- ❑ Exemple: stocker les notes (float) de 100 étudiants
- ❑ **Inconvénients**: Aucune possibilité de stocker des valeurs de différents types.

Exemple

- ❑ comment peut-on stocker les notes et le nom des étudiants?
 - ❑ Comment peut-on stocker des données de types différents selon nos besoins?
- ❑ Le langage C permet de faire quelque chose de très puissant: créer nos propres types de variables
- ❑ Le langage C propose **le concept des structures**: types de variables personnalisés.

Présentation générale

- ❑ Une **structure** est un assemblage de variables qui peuvent avoir **différents types**.
- ❑ Contrairement aux tableaux qui nous obligent à utiliser le même type dans tout le tableau, on peut créer une structure comportant des variables de types long, char, int.
- ❑ Créer de nouveaux types de variables devient indispensable quand on cherche à faire des programmes plus complexes.
- ❑ Une structure est un **objet** composé de plusieurs **champs** qui sert à représenter un objet réel ou un concept.
- ❑ Par exemple une voiture peut être représentée par les renseignements suivants : la marque, la couleur, l'année, etc.

Utilisation des structures de données

Définition

Structure de données

- ✓ Une structure est un enregistrement d'un objet composé d'une suite d'éléments en nombre fini comme les tableaux, mais dont les membres sont de types différents.

Exemple

Supposons qu'on désire stocker les informations de 2 personnes.

Il suffit de déclarer deux variables `pers1` et `pers2` de type **structure** et que chaque personne dispose de quatre informations :

- numéro : un entier
- taille : un réel
- poids : un réel
- sexe : un caractère.

<code>pers1</code>	"nom1"	1.75	67.9	'M'
<code>pers2</code>	"nom2 "	1.67	57.3	'F'

Déclaration et initialisation avec struct

- ❑ La déclaration d'une structure se fait en utilisant le mot-clé **struct** suivi du nom de la structure.
- ❑ Après les accolades, on définit les **membres/champs/attributs** de la structures avec leurs différents **types**.

Déclaration

```
struct nom_structure {  
  
    type1 nom_champs1;  
    type2 nom_champs2;  
    ...  
};
```

Exemple

```
struct Personne {  
    char nom[100];  
    float taille;  
    float poids;  
    char sexe;  
};
```

- ❑ Pour initialiser une variable avec le nouveau type, on réutilise le mot-clé **struct** suivi du nom de la structure.

Initialisation

```
struct nom_structure nom_variable ;
```

Exemple

```
struct Personne pers1 = {"nom1", 1.75, 64.7, 'M'};
```

Déclaration et initialisation avec typedef

- ❑ La répétition du mot-clé **struct** avec la première méthode est rapidement ennuyeuse.
- ❑ Il existe une méthode beaucoup plus simple et pratique qui consiste en l'utilisation du mot-clé **typedef**

Déclaration

```
typedef struct {  
  
    type1 nom_champs1;  
    type2 nom_champs2;  
    ...  
}nom_structure;
```

Exemple

```
typedef struct {  
    char nom[100];  
    float taille;  
    float poids;  
    char sexe;  
} Personne ;
```

- ❑ Lors de l'initialisation, il suffit de faire appel au nom de la structure.

Initialisation

```
nom_structure nom_variable ;
```

Exemple

```
Personne pers2 = {"nom2", 1.67, 57.3, 'F'};
```

Remarques

- ❑ La déclaration d'une structure se fait en dehors de toute fonction. Plus généralement, la déclaration d'une structure se fait dans un fichier séparé .h
- ❑ La déclaration d'une structure revient à déclarer un nouveau type et non à déclarer une variable. Lors de la définition d'une structure, aucun espace mémoire n'est réservé.
- ❑ On **DOIT** mettre un **point-virgule** après l'accolade fermante. C'est obligatoire. Si on ne le fait pas, la compilation plantera.
- ❑ Comme bonne pratique, le nom d'une structure commence par une majuscule afin de faire la distinction avec le nom d'une variable.
- ❑ Les structures peuvent être composées de champs de n'importe quel type : types de base, pointeur, tableau ou structure.
- ❑ Une structure ne peut pas comporter plus de 127 membres (champs).

Accès aux membres

- L'accès aux membres/champs d'une variable de type structure avec l'opérateur **point (.)** selon la syntaxe :

NomDeLaVariable.nomDuChamp;

```
typedef struct{
    char nom[100];
    float  taille, poids ;
    char  sexe ;
} Personne ;

Personne pers1 = {"nom2", 1.67, 67.8 , 'F'} ;
```



```
pers1.nom = "nom2" ;
pers1.taille = 1.67 ;
pers1.poids = 67.8 ;
pers1.sexe = 'F' ;
```

Exemple complet

Un programme qui lit sur clavier les informations d'une **personne p** et affiche par la suite ses caractéristiques.

```
#include <stdio.h>
typedef struct {
    char nom [40];
    float taille;
    int age;
} Personne;
int main () {
    Personne p;
    printf("Veuillez entrer le nom de la personne:");
    scanf("%s",p.nom);
    printf("Veuillez entrer la taille de la personne:");
    scanf("%f",&p.taille); /* ne pas oublier le & !!! */
    printf("Veuillez entrer l'âge de la personne:");
    scanf("%d",&p.age); printf("nom=%s\n",p.nom);
    printf("taille=%f\n",p.taille);
    printf("age=%d\n",p.age); }
```

Exercice

Écrire un programme qui demande à l'utilisateur d'entrer deux heures au format hh/mm/ss, puis qui les additionne et affiche le résultat. Dans ce but, vous devez :

1. Définir une structure de données **Horaire** permettant de représenter une heure au format : heure, minutes, secondes;
2. Ecrire une fonction qui permet d'afficher un **Horaire** sous la forme hh/mm/ss
3. Ecrire une fonction qui permet à un utilisateur de saisir un horaire.
4. Écrire une fonction qui permet de convertir un élément de type **Horaire** en un nombre de secondes (entier)
5. Écrire une fonction qui permet de convertir un nombre de secondes en un élément de type **Horaire**;
6. Écrire une fonction qui permet d'additionner deux éléments de type **Horaire**.

Corrigé – 1

1

```
//Définition de la structure de données
typedef struct {
    int heures;
    int minutes;
    int secondes;
}Horaire;
```

4

```
//Conversion d'un horaire en secondes
int conversion_sec(Horaire h){
    return h.heures*3600+h.minutes*60+h.secondes;
}
```

2

```
//Fonction d'affichage d'un horaire
void afficher(Horaire h){
    printf("%02d/%02d/%02d", h.heures,h.minutes,h.secondes);
}
```

3

```
//Fonction de saisie d'un horaire
Horaire saisir(){
    Horaire horaire;
    printf("Heures? "); scanf("%d",&horaire.heures);
    printf("Minutes? "); scanf("%d",&horaire.minutes);
    printf("Secondes? "); scanf("%d",&horaire.secondes);
    return horaire;
}
```

5

```
//Conversion d'un nombre de secondes en horaire
Horaire conversion_horaire(int sec){
    Horaire horaire;
    horaire.heures=sec/3600;
    horaire.minutes=(sec%3600)/60;
    horaire.secondes=sec%60;
    return horaire;
}
```

Corrigé – 2

6

```
//Formatage correct d'un horaire
Horaire formater(Horaire horaire){
    Horaire H;
    if(horaire.secondes>60){
        H.minutes=horaire.minutes+horaire.secondes/60;
        H.secondes=horaire.secondes%60;
    }
    if(horaire.minutes>60){
        H.heures=horaire.heures+horaire.minutes/60;
        H.minutes=horaire.minutes%60;
    }
    if(horaire.heures>=24) H.heures=horaire.heures%24;
    return H;
}
```

7

```
//Additionner 2 horaires
Horaire sum(Horaire h1, Horaire h2){
    Horaire somme;
    somme.heures=h1.heures+h2.heures;
    somme.minutes=h1.minutes+h2.minutes;
    somme.secondes=h1.secondes+h2.secondes;
    somme=formater(somme);
    return somme;
}
```

Manipulation des structures

Opérations sur les structures

Opérations globales sur les structures

- ❑ Contrairement aux tableaux, on peut affecter à une structure une autre structure d'un même modèle selon la norme ANSI. C'est la seule opération globale applicable aux structures.
- ❑ Exemple: **Personne pers2=pers1** ; signifie que les champs de la variable pers2 prendront les mêmes valeurs que ceux de la variable per1
- ❑ Il n'est pas possible de comparer deux structures. Les instructions `if(pers1 == pers2)` ou `if(pers1 != pers2)` ne sont pas permises
- ❑ L'opérateur **sizeof** permet de récupérer la taille d'un type structure ou d'une variable structure (`sizeof (struct Personne)` ou `sizeof (pers1 )`)

Opérations sur les membres des structures

- ❑ Les opérations possibles sur les champs d'un modèle structure sont les mêmes opérations applicables selon leur type (arithmétiques, comparaison). Exemple: **pers1.poids = pers2.poids*2;**

Structures et tableaux

- ❑ Si on a à traiter plusieurs modèles de la même structure, on peut donc optimiser le travail et créer un tableau du type de cette structure.
- ❑ Chaque élément du tableau représente donc un modèle de la structure que l'on a défini.

Exemple

```
Personne personnes[5];           //Chaque élément du tableau est un modèle de type Personne
personnes[1].taille=1.87;        //le champ taille du 2ème élément du tableau est initialisé à 1.87

for(int i=0; i<5; i++) {
    personnes[i].poids = 0.0;    //le champ poids de tous les éléments est initialisé à 0
}
```

- ❑ L'avantage d'utiliser un tableau, c'est de faire une boucle pour demander les informations de toutes les personnes à la fois sans avoir à répéter 5 fois le même code.
- ❑ Il suffit de parcourir le tableau '**personnes**' et de demander à chaque fois nom, taille, poids...

Structures et pointeurs

- ❑ S'il existe un objet d'un type, il doit être possible de créer un pointeur vers un objet de ce type.
- ❑ Un pointeur de structure se crée de la même manière qu'un pointeur de int, de double ou de n'importe quel autre type de base

```
Nom_structure* nom_pointeur;  
Personne* pers;
```

- ❑ L'utilisation d'un pointeur sur structure nécessite la gestion de l'accès à un membre via le pointeur.
- ❑ Intuitivement, on combinera sans doute les opérateurs ***** et **.** dans un nouvel opérateur flèche (->)

```
(*pers).taille = 1.67;
```

Equivalent à

```
pers -> taille = 1.67;
```

Structures et fonctions

- ❑ Ce qui nous intéresse dans l'utilisation des pointeurs sur structures est la possibilité d'utiliser le concept du passage par adresse.
- ❑ On fait passer un pointeur de structure à une fonction pour que celle-ci puisse modifier le contenu de la variable.

Passage par valeur

```
Personne modifier_taille(Personne pers) {  
    pers.taille = 1.80;  
    return pers;  
}
```

Passage par adresse

```
void modifier_taille(Personne* pers) {  
    pers -> taille = 1.80;  //équivalent à (*pers).taille  
}
```

Exercice

Programme de gestion des notes : A partir de la structure de données Etudiant suivante, créer les fonctions :

- ❑ Saisie() pour saisir les données ;
- ❑ Resultat() pour calculer la moyenne et attribuer la mention à un étudiant
- ❑ Afficher () pour afficher le relevé de note d'un étudiant.
- ❑ Le programme principal qui fait l'appel de ces fonctions

```
#include<stdio.h>
#define NBE 40 /* Nombre Maximum d'étudiants */
#define NBM 8 // Nombre Maximum des matières
typedef struct {
    char nom[100];
    double notes[NBM];
    double moyenne;
    char mention[15];
} Etudiant;
Etudiant Liste[NBE];
```

Structures imbriquées

- ❑ On peut déclarer un modèle de sous-structure comme un champ d'un modèle de structure principale.

Exemple

```
struct date {  
    int jour;  
    int mois;  
    int annee;  
};  
  
struct compte {  
    int no_compte ;  
    char nom[80];  
    float solde;  
    struct date dernierversement;  
};  
struct compte c1 = {12345,"Touzani",2000.45,01,11,2012};  
struct compte c[100];
```

Remarque : ordre de déclaration des structures

Accès aux membres :

c1.dernierversement.jour = 15;

c[12].dernierversement.mois = 11;

Énumérations

Définition d'une énumération

- ❑ Les énumérations constituent une façon un peu différente de créer ses propres types de variables.
- ❑ Une énumération ne contient pas de « sous-variables » comme c'était le cas pour les structures.
- ❑ Les énumérations permettent de créer un ensemble de **constantes** nommées ayant des valeurs entières.

Définition 1

```
typedef enum nom_énumération nom_type;  
enum nom_énumération  
{  
    VALEUR1, VALEUR2, VALEUR3,...  
};
```

Définition 2

```
enum nom_de_l_enum {  
    VALEUR1,  
    VALEUR2,  
    VALEUR3,  
    /* ... */  
};
```

Déclaration 1

```
Nom var = VALEUR;
```

Déclaration

```
enum Nom var = VALEUR;
```

Associer des valeurs

- ❑ Les valeurs possibles de l'énumération sont écrites en majuscules, tout comme les constantes et les `define`.
- ❑ Le compilateur associe automatiquement un nombre à chacune des valeurs possibles de l'énumération.

Exemple

```
enum Volume_Musique {  
    FAIBLE, MOYEN, FORT  
};
```

- ❑ Dans le cas de l'énumération `Volume`, `FAIBLE` vaut 0, `MOYEN` vaut 1 et `FORT` vaut 2.
- ❑ L'association est automatique et commence à 0. En fait, quand on a initialisé la variable `musique` à `MOYEN`, on a donc mis la case en mémoire à la valeur 1.
- ❑ Il est possible de demander d'associer une valeur précise à chaque élément de l'énumération au lieu des valeurs par défaut associées par le compilateur.

Exemple

- Dans cet exemple, la première constante "LUNDI" est assignée la valeur 1, et les constantes suivantes obtiennent des valeurs croissantes, avec "MARDI" obtenant la valeur 2, "MERCREDI" la valeur 3, et ainsi de suite.

Exemple

```
enum jours_de_la_semaine {  
    LUNDI = 1,  
    MARDI,  
    MERCREDI,  
    JEUDI,  
    VENDREDI,  
    SAMEDI,  
    DIMANCHE  
};
```

```
enum jours_de_la_semaine jour = LUNDI;  
if (jour == MARDI) {  
    printf("Aujourd'hui, c'est mardi.\n");  
}  
else{  
    printf(("C'est un autre jour.\n");  
}
```

Fin du chapitre 2
